Project 8.1 — Heart Disease Detection

Predicting Cardiovascular Disease from Patient Examinations · Capstone Learner Guide

This guide walks you through the project from start to finish. It tells you which path to take, gives you a problem statement to work from, a list of objectives to choose from, and the exact steps to follow.

1. The Story Behind The Data

Cardiovascular disease kills more people than anything else on earth. Most of the time it announces itself with a heart attack — by which point the useful window has closed.

The examinations that could catch it early are cheap: blood pressure, height and weight, a cholesterol reading, a few questions about lifestyle. Ten minutes and almost no equipment. The problem is not gathering the information. The problem is knowing which patients to worry about.

You are the analyst for a network of clinics. Your team wants a screening tool:

- From a routine examination, which patients are likely to have cardiovascular disease?
- Which measurements actually matter, and how much?
- Can we catch enough real cases to be worth using?
- Who does the tool miss — and how dangerous is that?

You have 70,000 patient examinations. This is real medical data, and the patterns in it are real.

> **Before anything else, understand what makes this project different.** Every other project in this module deals in dollars or minutes. This one deals in people. A wrong prediction here is not a bad estimate — it is a patient sent home who should have been tested. That single fact should shape your metrics, your threshold, and every recommendation you make. Section 3 and Step 8 explain how.

2. What Is In The Data

One file, `cardio_data.csv`. **70,000 rows, 16 columns, and not a single missing value.** Each row is one patient examination.

The target is perfectly balanced: **50% of patients have the disease, 50% do not.** That is unusual and it is helpful — it means accuracy is a reasonable metric here, unlike most medical datasets. But it is still not the *right* metric. See Step 8.

Column	What it is
`disease`	**Your target.** 1 = has cardiovascular disease, 0 = does not.
`age`	Age — **in days, not years**. See Step 2.
`gender`	1 or 2. The dictionary does not say which is which. **You will have to work it out** — see Step 2.
`height`	Height in **centimetres**. The dictionary says "DAYS" — that is a typo. Trust the data.
`weight`	Weight in kilograms.
`ap_hi`	Systolic blood pressure (the top number). **Badly corrupted — see Step 3.**
`ap_lo`	Diastolic blood pressure (the bottom number). **Also badly corrupted.**
`cholesterol`	1 = normal, 2 = above normal, 3 = well above normal. **Ordered**, not just categories.
`gluc`	Glucose, same 1–2–3 scale.
`smoke`, `alco`, `active`	Yes/no flags for smoking, alcohol and physical activity.
`id`	Patient identifier. **Never use this as a feature.**
`date`, `country`, `occupation`	Examination date, country, job. **Read Step 4 before you use these.**

What this data can and cannot tell you

The medical columns are real and they work. Cholesterol level moves the disease rate from 44% at normal, to 60% at above-normal, to **76.5%** at well-above-normal. Blood pressure is the single strongest predictor once cleaned. Age matters. These are genuine medical relationships, not artefacts — which makes this one of the most rewarding datasets in the module.

But three columns are decorative. `country`, `occupation` and `date` were added on top of the original medical data and carry **no signal whatsoever**. Every country has a disease rate of about 50%. Every occupation has a disease rate of about 50%. Doctors, chefs, lawyers and nurses are all identical.

This is worth proving rather than assuming — see objective 12. Testing a plausible-sounding feature and showing it contributes nothing is real analysis, and it stops you building a dashboard page about "disease by occupation" that shows six identical bars.

3. Your Path: Predictive Analytics

Build a model that predicts cardiovascular disease. You will train a classifier in Python that takes a patient's examination results and returns the probability that they have the disease.

Why this is the right choice for this data

- **The target is handed to you.** `disease` is right there, labelled, on all 70,000 rows. The data dictionary itself calls it the target variable. This dataset exists to be modelled.
- **70,000 rows is plenty.** More than enough to train, validate and test properly.
- **The features genuinely drive the outcome.** Blood pressure, cholesterol and age are what a cardiologist would actually ask about. Your model has a real chance of working.
- **The classes are balanced.** A 50/50 split removes the imbalance headache that ruins most medical modelling projects.
- **The end product is genuinely useful.** A screening tool that flags patients for further testing is something a clinic would actually use.

Follow this guide and build the model.

4. Your Problem Statement

> **"To build a screening model that estimates the probability of cardiovascular disease from routine examination measurements — blood pressure, cholesterol, age, body measurements and lifestyle — so that clinics can prioritise which patients to refer for further testing."**

You may use this as it is, or adjust it. If you change it, keep the same four parts:

1. **What you are building** — a screening model.
2. **What it uses** — routine examination measurements.
3. **Who it is for** — clinics and their doctors.
4. **What decision it helps with** — who to refer for testing.

> **Say "screening", not "diagnosis".** Your model flags patients who may need a doctor's attention. It does not diagnose anyone, and it does not replace a clinician. Framing it correctly is not a disclaimer — it is the difference between a tool a hospital could use and one no regulator would allow near a patient. State this plainly on your first slide.

You must get this approved by your trainer before you start building (Sessions 3–4).

5. Choose Your 5 Objectives

You need at least five research objectives. Below are fifteen you can choose from. **Pick five.**

One tip: take two about risk factors, two about model quality, and one from the third group. Objective 7 is the most important on this list for a medical project.

What drives the disease

#	Objective	Why it matters
1	Identify which measurements most strongly predict cardiovascular disease, and rank them.	The core question. Blood pressure will lead — but by how much over cholesterol and age?
2	Quantify how the disease rate changes across the three cholesterol levels.	Strong and clean. The rate climbs from 44% to 76.5% — a genuine medical finding.
3	Measure how disease risk rises with age.	Requires converting age from days first. Expect a steady climb.
4	Measure the relationship between BMI and disease, after engineering BMI from height and weight.	BMI is not in the data. You have to build it — and it is one of the most useful things you can add.
5	Compare disease rates across blood pressure categories using real clinical thresholds.	Turning numbers into Normal / Elevated / Stage 1 / Stage 2 makes your findings speak a doctor's language.

How good is the model

#	Objective	Why it matters
6	Compare your model against a baseline that predicts the majority class.	With a 50/50 split the baseline scores 50%. Everything you build must beat it clearly.
7	Measure the trade-off between recall and precision, and recommend a threshold for clinical screening.	**The most important objective here.** A missed case is not the same as a false alarm. See Step 8.
8	Compare a logistic regression against a tree-based model on the same data.	In medicine, a model a doctor can follow may be worth more than a slightly better black box.
9	Identify which patients the model misclassifies, and find what they have in common.	Excellent objective. Knowing who your tool fails is a safety requirement, not a nicety.
10	Determine the smallest set of measurements that still gives useful screening accuracy.	A tool needing three measurements gets used. One needing ten does not.

Data quality and honest findings

#	Objective	Why it matters
11	Quantify the impossible values in the blood pressure columns and their effect on any analysis.	This data contains a systolic reading of 16,020. Documenting the damage is real work.
12	Test whether country and occupation add any predictive value.	They do not. Proving it is worth more than assuming it.
13	Investigate why smokers show a *lower* disease rate in this data.	**Fascinating and important.** The data really does say this. Work out why before you present it — see Step 4.
14	Compare your model against a simple clinical rule based on blood pressure, cholesterol and age.	If a three-line rule matches your model, that is a finding — and an honest one.
15	Measure how much accuracy is lost by excluding blood pressure entirely.	Tests how much of the answer rests on one measurement.

6. The Steps To Follow

Step 1 Look at the data first

Load the file and run `.info()`, `.head()`, `.describe()` and `.shape`.

You should be able to answer these before moving on:

- How many rows and columns? Any missing values?
- What is the balance of the `disease` column?
- **Look hard at `.describe()` for `ap_hi` and `ap_lo`.** What are the minimum and maximum? Is the standard deviation bigger than the mean? What does that tell you?
- What is the range of `age`? Does 18,393 look like an age to you?

Step 2 Fix the units and work out the codes

Age is in days. A value of 18,393 means about 50 years old. Convert it: `age_years = age / 365.25`. The result runs from about 30 to 65 — which makes sense for a cardiovascular study. Leaving it in days will not break your model, but every chart and every coefficient you present will be unreadable.

Gender is 1 or 2, and nothing tells you which is which. Do not guess. Work it out from the data — group by gender and compare average height and smoking rate. One group averages about 161cm with a 2% smoking rate; the other averages about 170cm with a 22% smoking rate. That settles it. **Put this reasoning in your presentation** — it shows you investigated rather than assumed.

Cholesterol and glucose are ordered, not unordered categories. 1 is better than 2, which is better than 3. Do not one-hot encode them and throw that ordering away — leave them as numbers, or use an ordinal encoding.

Step 3 Clean the blood pressure

This is the most important step in the project. Look at what is actually in these columns:

Measure	ap_hi (systolic)	ap_lo (diastolic)
Minimum	**−150**	**−70**
Maximum	**16,020**	**11,000**
Mean	128.8	96.6
Standard deviation	154.0	**188.5**
Median	120	80

Read the diastolic column again: mean 96.6, standard deviation 188.5. The spread is twice the average. A real diastolic reading sits between about 60 and 120. Nobody has ever had a blood pressure of 16,020 — a human being with a systolic reading of 370 is already in a crisis.

These are typing errors. Someone recorded 160/20 as `16020`. Someone else added a zero and turned 110 into `1100`. And **1,234 patients have a diastolic reading higher than their systolic** — which is physically impossible.

Leave these in and your averages are meaningless, your scatter plots are a single dot with a few specks in the distance, and your model spends its effort fitting nonsense.

Filter to values a human could actually have:

- **Systolic (`ap_hi`) between about 60 and 250.** Removes 7 negatives, 40 above 300, and 188 below 60.
- **Diastolic (`ap_lo`) between about 30 and 200.** Removes 953 above 200 and 53 below 30.
- **Systolic must be greater than diastolic.** Removes 1,234 impossible pairs.

About **68,678 rows survive — 98.1% of your data**. You lose almost nothing and gain a dataset that means something. Once cleaned, systolic blood pressure correlates with disease at 0.43, making it your strongest single predictor. Before cleaning, that signal is buried.

Also check `height` and `weight`: the data contains adults recorded at 55cm tall and 10kg. Decide what to do and say what you did.

Count what you remove and say why. "We excluded 1.9% of records: 1,234 had a diastolic reading above their systolic, which is physically impossible; 953 had diastolic readings above 200..." is a professional statement. It also directly answers objective 11.

Step 4 Work out which columns actually matter

Before modelling, test your columns rather than trusting them.

Country, occupation and date are noise. Group the disease rate by each and look: every country sits at about 50%, every occupation at about 50%. They tell you nothing. Leave them out of the model and say why.

The smoking paradox — do not skip this.

Check the disease rate for smokers against non-smokers. You will find that **smokers appear *less* likely to have cardiovascular disease** (about 47.5% against 50.2%). The same happens with alcohol.

This is not a data error, and it is certainly not a medical finding. Smoking causes heart disease — that is one of the most firmly established facts in medicine. What you are seeing is **confounding**: smokers in this dataset differ from non-smokers in other ways (age and sex, for a start), and those differences swamp the effect of smoking itself.

Never present this as "smoking protects your heart." Present it as a demonstration that a raw comparison between two groups can point the wrong way when a third factor is driving both. Investigating this properly — compare smokers and non-smokers *within the same age band and sex* — is objective 13, and it is the most intellectually impressive thing you can do with this dataset.

Step 5 Engineer your features

New feature	How to build it
BMI	`weight ÷ (height/100)²`. **The most valuable feature you can add.** Neither height nor weight means much alone — together they mean a great deal.
age_years	`age ÷ 365.25`. Essential for readable output.
pulse_pressure	`ap_hi − ap_lo`. A real clinical measure of arterial stiffness. Almost nobody will think of this.
bp_category	Group blood pressure into Normal, Elevated, Stage 1, Stage 2 using clinical thresholds. Makes your charts speak a doctor's language.
bmi_category	Underweight / Normal / Overweight / Obese, using the standard WHO cut-offs.

Drop `id`. It is a row number. A model will happily use it and learn nothing real.

Step 6 Split and train

A random 80/20 split is fine here — this is not time-series data, and each row is a separate patient. Use `stratify=y` to keep the 50/50 balance in both halves.

1. **Baseline first.** Predict the majority class for everyone. With a balanced

target that scores 50%. Record it — it makes every later number meaningful.

2. **Logistic Regression.** Fast, and each coefficient states how much a risk factor shifts the odds. Doctors understand this kind of model, which matters more here than in any other project.
3. **Random Forest**, then optionally XGBoost. Usually stronger, because risk does not rise in a straight line with blood pressure.
4. **Wrap it in a `Pipeline`** holding your scaling and model together. It prevents leakage and becomes what you deploy in Step 9.

Step 7 Evaluate like a clinician, not a competitor

> **The two mistakes your model can make are not equal, and this changes everything.**
>
> A **false positive** means a healthy patient gets referred for a test they did not need. Cost: some worry, some money, an hour of a doctor's time.
>
> A **false negative** means a patient *with* cardiovascular disease is told they look fine and sent home. Cost: potentially their life.
>
> These are not the same, so **do not optimise for accuracy**. Accuracy treats both errors as identical. For a screening tool, **recall** — the share of genuinely sick patients your model catches — is the number that matters most.

Report all of these:

Metric	What it tells you
Recall	Of patients who have the disease, how many did we catch? **Lead with this.**
Precision	Of the patients we flagged, how many really had it? Governs how much unnecessary testing you cause.
F1	The balance of the two.
ROC-AUC	Overall separating power, independent of threshold.
Confusion matrix	Show it. Point at the false negatives and say what that number means in human terms.
Accuracy	Meaningful here because the classes are balanced — but never your headline.

Then tune the threshold. The default 0.5 is an arbitrary convention, not a

medical decision. Lower it to 0.4 and you catch more sick patients while referring more healthy ones. That trade-off is a clinical judgement, and presenting it as a choice — with the numbers attached — is exactly what objective 7 asks for and exactly what a real health service would need.

Step 8 Interpret the model

Interpretability is worth 20% of the predictive rubric, and in medicine it matters more than the percentage suggests. A doctor will not act on a number with no reason attached.

- Show **feature importance** or logistic regression coefficients.
- Translate them into plain language: "each 10-point rise in systolic pressure raises predicted risk by roughly X%."
- Use **SHAP** if you can — it explains an individual patient's score, which is exactly what a doctor would ask about.
- Check your findings against known medicine. Blood pressure, cholesterol and age leading is a good sign. If something strange tops your list, investigate before you present it.

Step 9 Deploy, scale and monitor

Deploy it. Save your pipeline with `joblib`, then build a small Streamlit or Gradio form — age, height, weight, blood pressure, cholesterol, glucose, lifestyle — returning a risk score and a clear recommendation. Label it a screening aid, not a diagnosis. It takes an afternoon and demos powerfully.

Think about scale. A clinic network runs thousands of examinations a month. Explain batch scoring, the pipeline handling new data identically, and what retraining costs.

Plan the monitoring. Say what you would watch: recall against confirmed diagnoses as they come back, drift in the patient population, changes to clinical thresholds. Name the recall level that would trigger a retrain — and who is accountable for checking.

Step 10 Prepare your presentation

- **2 minutes** — the problem, and the words "screening tool, not a diagnosis".
- **3 minutes** — your data, and the blood pressure cleaning.
- **4 minutes** — what drives the disease, and by how much.
- **3 minutes** — your model, led by recall, with the threshold trade-off.
- **3 minutes** — a live demo of the screening tool.

Practise with a timer. Running over time is the easiest way to lose marks.

7. Before You Submit

Bundle everything into one ZIP file for the Practical Performance assessment: your notebooks, your saved model, the dataset, your app code, and anything else. Upload your presentation (.pptx) separately. Both must be in before your presentation session.

> **One last thing.** If you take one idea from this project, take this: your model's false negatives are patients. When you show the confusion matrix, do not just read out the number in the bottom-left cell — say what it means. "Our model missed 312 patients who have the disease" is the sentence that separates someone doing an exercise from someone doing the work.

Capstone Learner Guide · Project 8.1 — Heart Disease Detection
Dataset: cardio_data.csv · 70,000 patient examinations, 16 columns, no missing values · Target: `disease` (50/50 balanced)